

Replace string in multiple files using sed

Author: Marcos de Carvalho

Date: 2026-04-21

Replaces all occurrences of a string with a replacement string in multiple files or file patterns. Pure oneliner that works directly pasted with parameters, or in an alias.

Language

bash

Category

text-processing

Command

```
bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec  
sed -i "s#$old#$new#g" {} +' _
```

Explanation

This oneliner uses a subshell ('bash -c') to parse positional parameters. The first argument is the string to replace ('\$1'), the second is the replacement ('\$2'), and 'shift 2' leaves the remaining arguments as the files or patterns to search in ('\$@'). The 'find' command locates all matching files and processes them with 'sed' using the '-exec {} +' syntax, which efficiently handles multiple files in batches. The '_' at the end acts as '\$0' (the script name) so '\$1' maps to the first actual argument. This approach handles an arbitrary amount of files and supports wildcard file patterns without hardcoding values into the command itself.

Tags

sed, text-processing, string-replacement, file-editing, search-replace, batch-replace

Dependencies

sed, coreutils

Arguments

1. **SEARCH** (Required): The string to search for (first parameter passed after the command).
2. **REPLACE** (Required): The replacement string (second parameter passed after the command).
3. **FILES** (Required): One or more file paths or patterns (e.g., file1.txt, *.txt, /path/*.js).

Examples

1. `bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' - 'old-domain.com' 'new-domain.com' config.json settings.json` - Paste directly: replace 'old-domain.com' with 'new-domain.com' in config.json and settings.json
2. `bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' - '/var/old/path' '/var/new/path' nginx.conf` - Replace a path containing slashes in a configuration file
3. `bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' - 'old-text' 'new-text' *.txt` - Replace in all .txt files in current directory using a wildcard
4. `alias repl="bash -c 'old=\{}$1; new=\{}$2; shift 2; find \{}"\{}$@\{}" -type f -exec sed -i \{}"s#\{}$old#\{}$new#g\{}" {} +' -"` - Create an alias named 'repl' to easily perform string replacements
5. `repl 'deprecated.method' 'new.method' src/**/*.py` - Use the alias to replace in all Python files in src directory and subdirectories
6. `repl 'old-api' 'new-api' config/*.json docs/*.md` - Use the alias to replace in multiple file types across different directories

Output

(No output on success - files are modified in place)

Notes

- Pure oneliner: uses `bash -c` to accept arbitrary parameters after the command string

- Usage: Paste the command and append your search string, replacement string, and file patterns
- Can be easily turned into a powerful alias in your `.bashrc` or `.bash_profile`
- The `-i` flag modifies files in place; make backups first with `cp file.txt file.txt.bak` if concerned
- Using `#` as delimiter prevents issues when search/replace strings contain forward slashes
- The `g` flag replaces all occurrences per line; remove it to replace only first occurrence
- The `-exec {} +` syntax bundles multiple files to `sed` for efficiency
- Files with spaces or special characters are handled correctly by `find`
- For case-insensitive replacement, modify the command to include the `i` flag: `sed -i "s#$old#$new#gi"`

Warnings

- The `-i` flag modifies files directly without confirmation - always test on copies first
- If search string is empty, `sed` will replace every character with the replacement string
- The command will silently fail to modify files if they lack write permissions
- Very large files or many files may cause performance issues
- Replacement is literal - regex special characters will be treated as patterns (not escaped)
- Be careful with wildcard patterns - they may match more files than intended
- The `find` command only processes regular files, not directories
- Special characters in search/replace strings must be properly quoted or escaped

See Also

- `find-large-files-recursive`
- `disk-space-usage-per-directory`

Status

reviewed

Safety

caution

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-21

Updated At

2026-04-21